ISLA 4

Equipo 4 · Categoría: Cafetería

2ª Entrega — Proyecto Integrador (PI)

P

Priscilla

M

Melissa

J

Juan

L

Luis

1 Protocolo de mensajes

Protocolo interno de la Isla 4 — texto plano, máx. 256 bytes, cada campo validado con regex

Formato general

```
ORIGEN|DESTINO/TIPO/campo1/campo2/...
```

IDs: CLI_04 · INT_04 · SERV (bodega). El tipo va en dos dígitos.

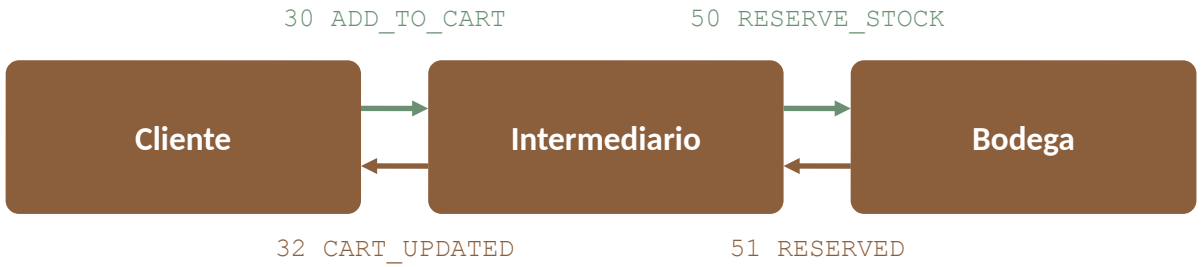
Mensajes

`10 → 11`	categorías disponibles
`20 → 22 · 23`	productos de una categoría (o vacía)
`30 → 32`	agregar al carrito
`40 → 41 · 42`	factura proforma (o carrito vacío)
`50 → 51`	reserva de stock en la bodega

Errores del protocolo

`90 ERR_FORMAT`	campo no cumple la regex
`91 ERR_SIZE`	campo fuera del tamaño permitido
`92 ERR_COMM`	un componente no respondió a tiempo

Flujo de una petición — agregar al carrito (30)



Idea clave: el Intermediario valida los campos antes de molestar a la Bodega, y arma la factura (41) con lo que ya se confirmó en cada 51, sin volver a consultarla.

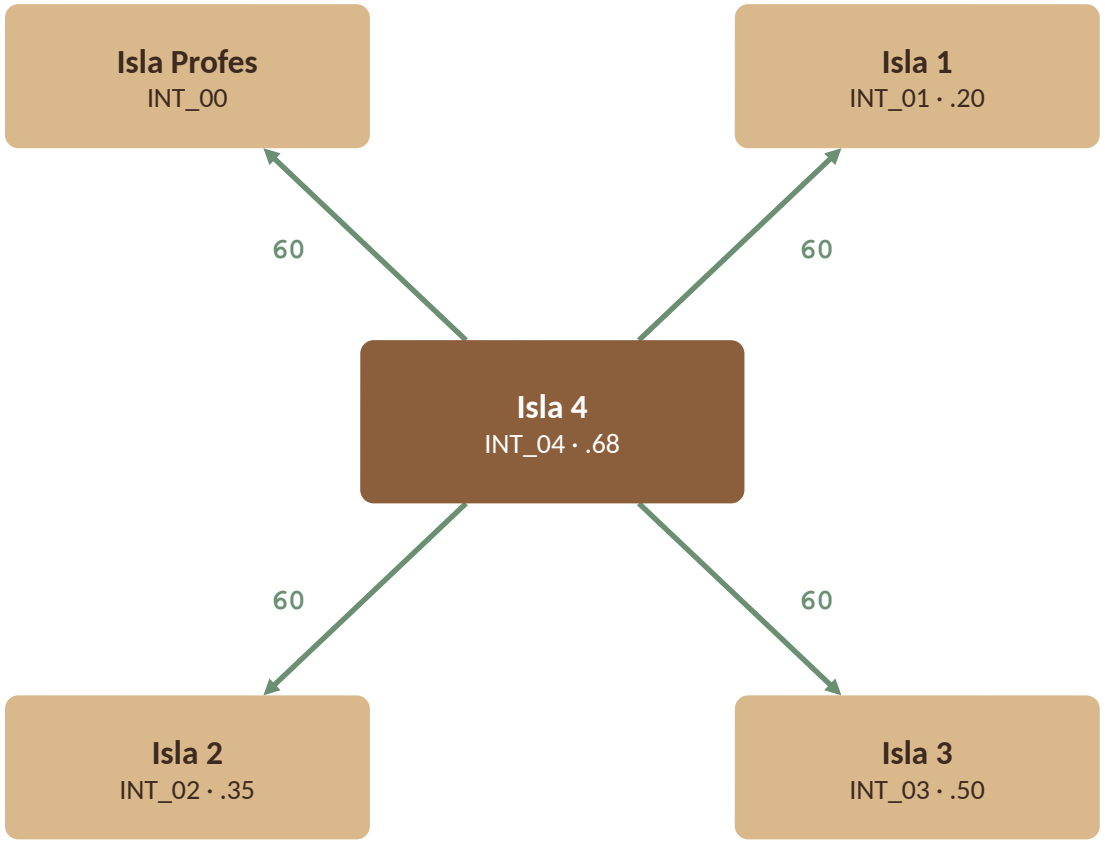
Ejemplo de log

```
CLI_04|INT_04/30/Flan-caramelo/2
INT_04|SERV/50/Flan-caramelo/2
SERV|INT_04/51/Flan-caramelo/3.20/2
INT_04|CLI_04/32/Flan-caramelo/3.20/2
```

2

Protocolo mancomunado

Borrador entre islas: descubrimiento por UDP y datos por TCP — se implementa en la siguiente etapa



Al arrancar, la isla 4 manda 60/INT_04/172.16.123.68/5353 y cada isla le responde con su propio 60.

Mensajes entre intermediarios

`60/id_isla/ip/puerto`	ANNOUNCE: presencia y dirección (UDP)
`61/id_isla`	DEATH: la isla avisa que se cierra
`01`	ACK: deja la conversación en espera
`10 · 20 · 50`	categorías, productos y reserva
`40 → 41`	factura del pedido

Puertos

`5353`	UDP · ANNOUNCE por broadcast
`9090`	TCP · datos entre intermediarios

Formato

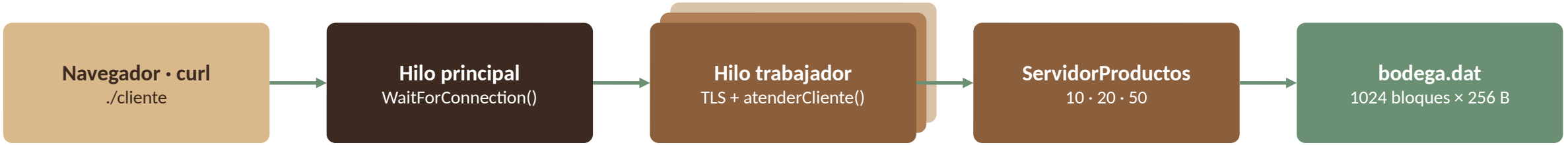
`TIPO/campo1/campo2/...`

Sin ORIGEN|DESTINO: el emisor se reconoce por la IP y el puerto del socket. Máx. 256 bytes.

3

Server.cc — servidor de productos

HTTPS con un hilo por conexión: el hilo principal solo acepta, los trabajadores atienden



GET o POST por HTTPS

solo acepta, nunca atiende

uno por cliente, con try/catch

reparte entre las bodegas

catálogo y stock en disco

GET → catálogo en HTML

Se abre desde el navegador de una computadora o del celular, y con curl -k.

POST → protocolo

El cuerpo de la petición lleva el mensaje, por ejemplo CLI_04|INT_04/20/postres.

El hilo principal solo acepta

```
int fd = s1->WaitForConnection();
std::thread([fd, s1, &bodega]() {
  auto c = s1->CompletarConexion(fd);
  atenderCliente(c, &bodega);
}).detach();
```

Un cliente no tumba al servidor

Si una conexión falla (TLS o Content-Length inválido) se descarta y el servidor sigue.

Carrito por cliente

Guardado con mutex por ID de origen; la factura sale de lo ya reservado.

12 / 12

casos de test_flujo.sh pasan: cliente, curl, mayúsculas, stock insuficiente, ERR_COMM y el servidor sigue vivo al final.

4

Client.cc — cliente HTTPS

De la terminal al servidor: cada comando viaja como un POST cifrado con el mensaje del protocolo

1

main()

lee el comando de la terminal (10, 20, 30, 40 o error) y decide a qué función delegar

2

pruebaCategorias / pruebaListCat / pruebaAgregarCarrito / pruebaFactura

arman el mensaje del protocolo con construirMensaje()

3

enviarAlIntermediario()

construirPeticionHTTP() arma un POST /mensaje con el mensaje en el cuerpo

4

SSLSocket

Connect(), Write() y Read() viajan cifrados con TLS

5

extraerCuerpoRespuesta() + parsearMensaje()

separan el cuerpo HTTP y muestran categorías, productos o la factura

enviarAlIntermediario()

```
client = new SSLSocket(false);
client->Connect(host, puerto);

peticion = construirPeticionHTTP(
    "/mensaje", host, msg);
client->Write(peticion.c_str());
resp = leerRespuestaCompleta(client);
```

Carrito local

El cliente recuerda lo que el servidor confirmó en cada 32 y con eso pide la factura: 40/count/lista.

Avisa si la reserva vino en 0 o incompleta. El comando error fuerza un ERR_COMM.

5

Sistema de archivos — bodega.dat

1024 bloques de 256 bytes (256 KB) en un solo archivo, con varias bodegas adentro



14 bloques usados de 1024 · quedan 1010 libres para crecer

2 bodegas · 4 categorías · 15 productos

Bodega1	`postres`	`caramelos`
Bodega2	`bebidas`	`reposteria`

Directorios encadenados

Cada bloque de directorio tiene 7 casillas y un encabezado; al llenarse se pide otro bloque y se enlaza con siguienteBloque.

Un bloque índice por categoría

Apunta a los bloques de datos de sus productos, con 4 registros por bloque.

Registro de producto — 64 bytes fijos

`nombre`	`56 B`	hasta 50 caracteres
`cantidad`	`2 B`	uint16, stock en bodega
`precio`	`4 B`	uint32, en centavos
`estado`	`1 B`	0 libre / 1 ocupado

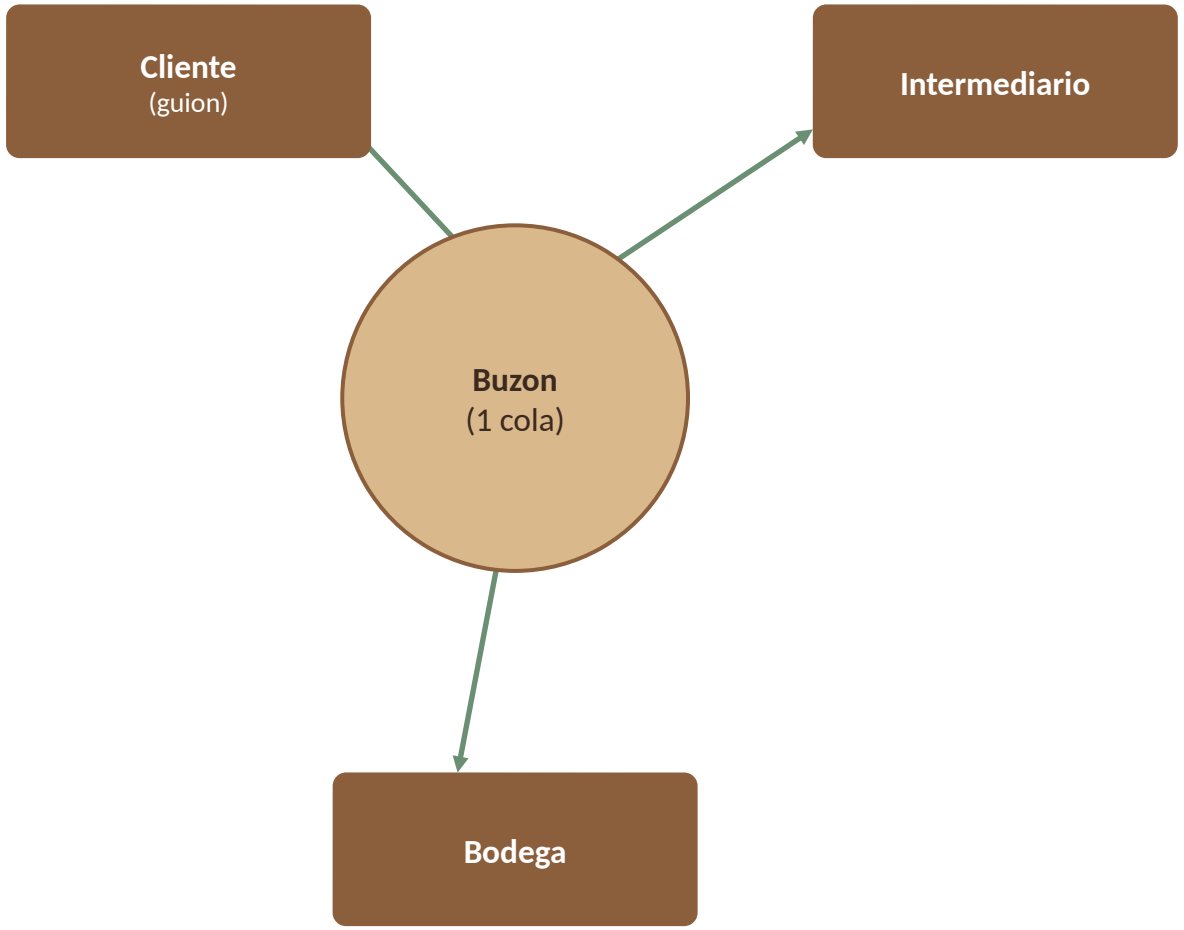
En la etapa 1 el registro era de 32 B (24 para el nombre); se amplió a 64 B para aceptar los 50 caracteres del protocolo.

`make_fileSys` · `read_fileSys` · `fileSys_herr`

formatean, leen y editan bodega.dat desde la terminal

6 Simulación (IPC + hilos)

El mismo Buzon System V de la etapa 1, ahora hablando el protocolo interno completo



`mtype = origen × 10 + destino  →  cada hilo recibe solo su canal`

1 Formato del protocolo interno

armarMensaje() y parsearMensaje(): ORIGEN|DESTINO/TIPO/campos.

2 Carrito con reserva

30 → 50 → 51 → 32: la bodega reserva lo que tiene (pide 500, reserva 80).

3 Factura proforma

40 → 41 con subtotales y total; 42 si el carrito está vacío.

4 Filtro por categoría y errores

20 devuelve solo esa categoría; 90 por regex y 92 si la bodega no responde.

Log de la simulación — reserva parcial

```
[CLI_04 -> INT_04] CLI_04|INT_04/30/fresa/500
[INT_04 -> SERV  ] INT_04|SERV/50/fresa/500
[SERV   -> INT_04] SERV|INT_04/51/fresa/350.00/80
[INT_04 -> CLI_04] INT_04|CLI_04/32/fresa/350.00/80
```

7 Packet Tracer — laboratorio 3-5

La isla 4 sale a la red del laboratorio por un troncal 802.1Q y llega a las demás islas



red 172.16.123.64 /28 · gateway .65 · VLAN 240 (Equipo4)

Qué se hizo

Se configuró Fa0/24 como troncal 802.1Q para llevar la VLAN 240 al resto del laboratorio. Desde la PC del equipo se hizo ping a la isla 1 y respondieron los 4 paquetes; el TTL de 127 muestra que el tráfico pasa por un router.

Gateway: .65
Switch (Vlan240): .66
PC Equipo 4: .68

Configuración clave

```
hostname Isla-Equipo4

vlan 240
 name Equipo4

interface range Fa0/1 - 2
 switchport mode access
 switchport access vlan 240

interface Fa0/24
 switchport mode trunk

interface Vlan240
 ip address 172.16.123.66
            255.255.255.240
```

Verificación en la terminal

show interfaces trunk

```
Isla-Equipo4#show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Fa0/24      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Fa0/24      1-1005

Port        Vlans allowed and active in management domain
Fa0/24      1,240

Port        Vlans in spanning tree forwarding state and not pruned
Fa0/24      1,240
```

ping 172.16.123.20 (isla 1)

```
C:\>ping 172.16.123.20

Pinging 172.16.123.20 with 32 bytes of data:

Reply from 172.16.123.20: bytes=32 time<1ms TTL=127
Reply from 172.16.123.20: bytes=32 time<1ms TTL=127
Reply from 172.16.123.20: bytes=32 time<1ms TTL=127
Reply from 172.16.123.20: bytes=32 time<1ms TTL=127

Ping statistics for 172.16.123.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
```